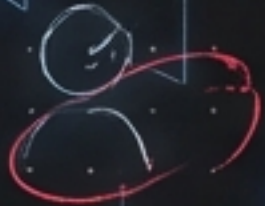
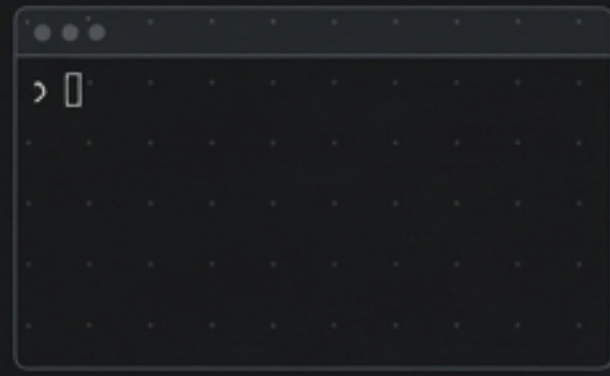


Case Studies in Software Architecture

Translating human ambiguity into system logic.



```
File Code = {  
  ...  
  public void test() {  
    ...  
    testOfUnsubscribed();  
  }  
}
```

CSE322 :: Software Engineering Workshop

EDGE
CASE



SECURITY
RISK



INTERACTIVE
PROMPT



Focus: Safe Home Access System

Failure of Assumptions in Design

Human Assumption



How a beginner writes a system requirement: "Just wash your hands."

Algorithmic Reality

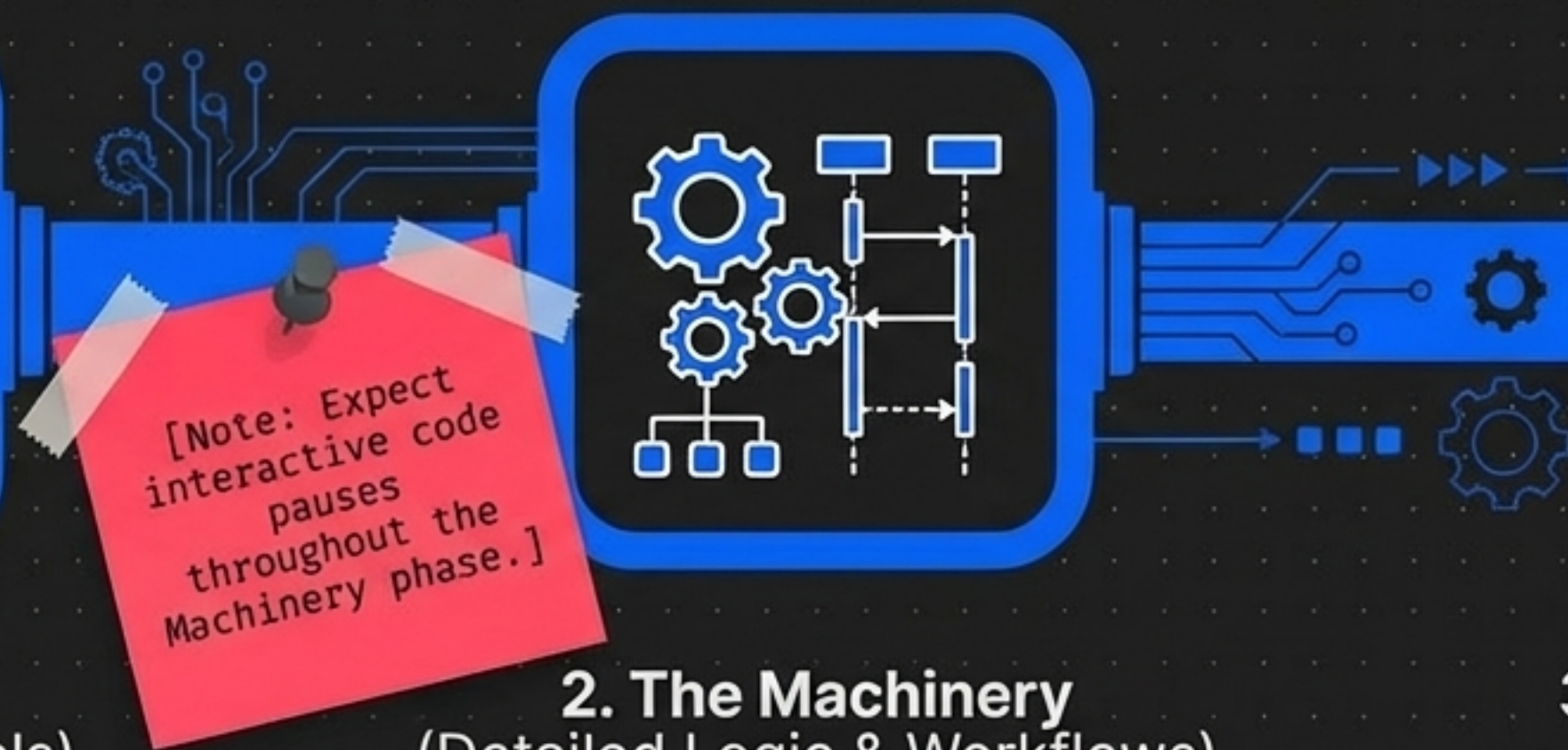


What is obvious to a human is **invisible to a machine**.
95% of humans fail to execute this algorithm correctly.
Machines require **absolute explicitness**.



1. The Blueprint (Defining Actors & Goals)

Extracting entities from
ambiguous requirements.



2. The Machinery (Detailed Logic & Workflows)

Mapping system states,
sequences, and error handling.



3. The Human Interface (UI & Edge Cases)

Economizing design
through constraint.

Phase 1: Raw Data vs. Extracted Goals

Raw Requirements

REQ1: Keep door locked and auto-lock
REQ2: Lock when 'LOCK' pressed
REQ3: Unlock when valid key provided
REQ4: Allow mistakes but prevent dictionary attacks
REQ5: Maintain a history log
REQ6: Adding/removing users at runtime
REQ6: Adding/removing users at runtime
REQ7: Configuring the device activation preferences
REQ8: Inspecting the access history

Extracted Goals

Unlock and enter home →
Unlock (UC-1)

Create new user account
→ AddUser (UC-3)

INITIATING ACTORS (PRIMARY)

Defines the actor who initiates the use case to achieve a goal.



TENANT



LANDLORD

SUPPORTING ACTORS (SECONDARY)

Defines the actor who helps the system-to-be complete the use case.



LOCKDEVICE



LIGHTSWITCH



ALARMBELL

OFFSTAGE ACTORS

Defines actors that passively participate or are merely notified.



TIMER



POLICE



DATABASE

NOTICE: "HOME" AND "DOORS" ARE NOT ACTORS. ONLY ENTITIES THAT INTERACT WITH THE SYSTEM BOUNDARY QUALIFY.

Strategic Traceability Heatmap

	UC1	UC2	UC3	UC4	UC5	UC6	UC7	UC8
REQ1	X	X						
REQ2		X						
REQ3	X						X	
REQ4	X						X	
REQ5	X	X						
REQ6			X	X				X
REQ7						X		X
REQ8					X		X	
REQ9					X		X	
Total PW	15	3	2	2	3	9	2	3

Highest
Priority

Lowest
Priority

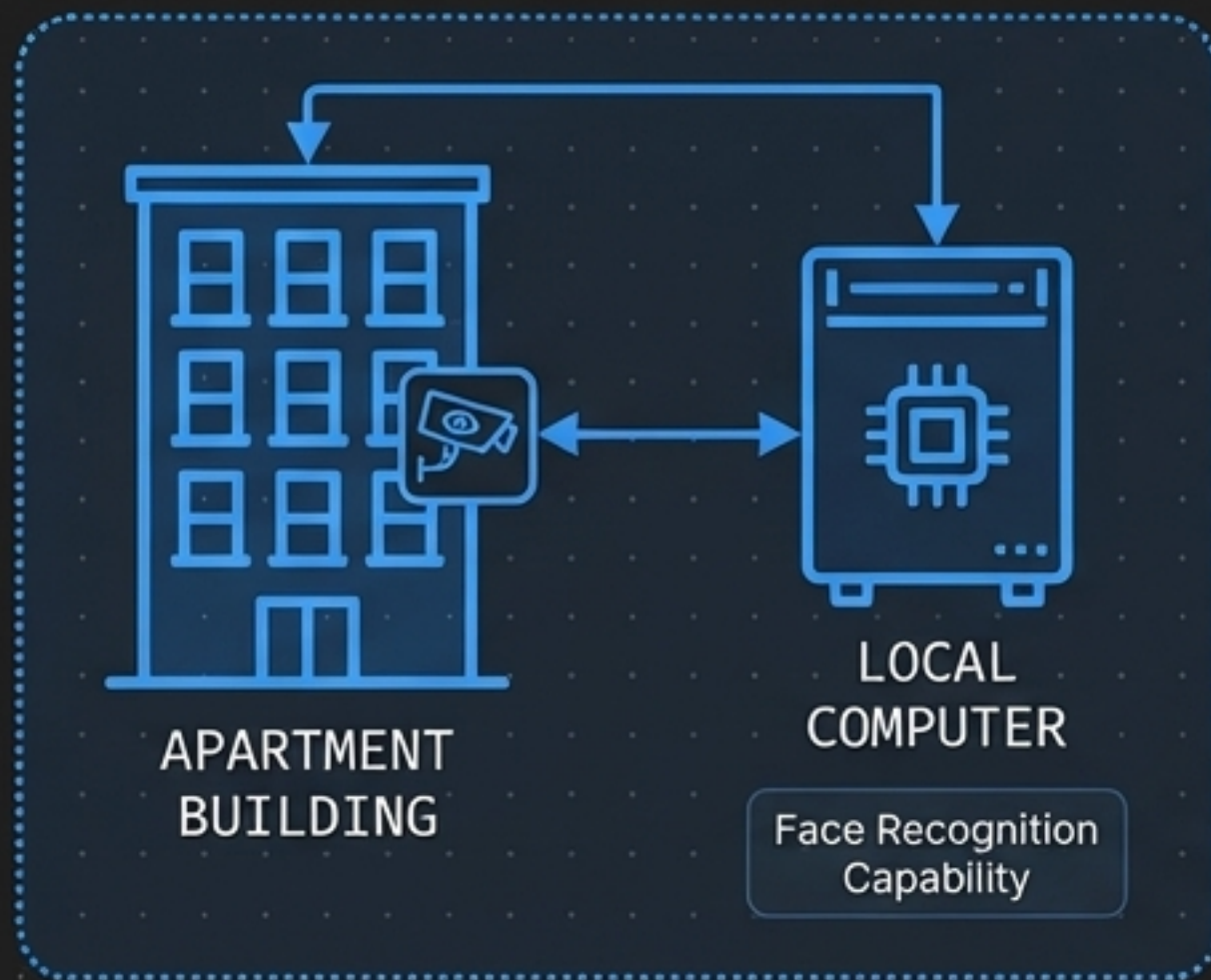
Key Takeaway

Traceability ensures no orphaned requirements, no unjustified features, and mathematically prioritizes engineering sprints.

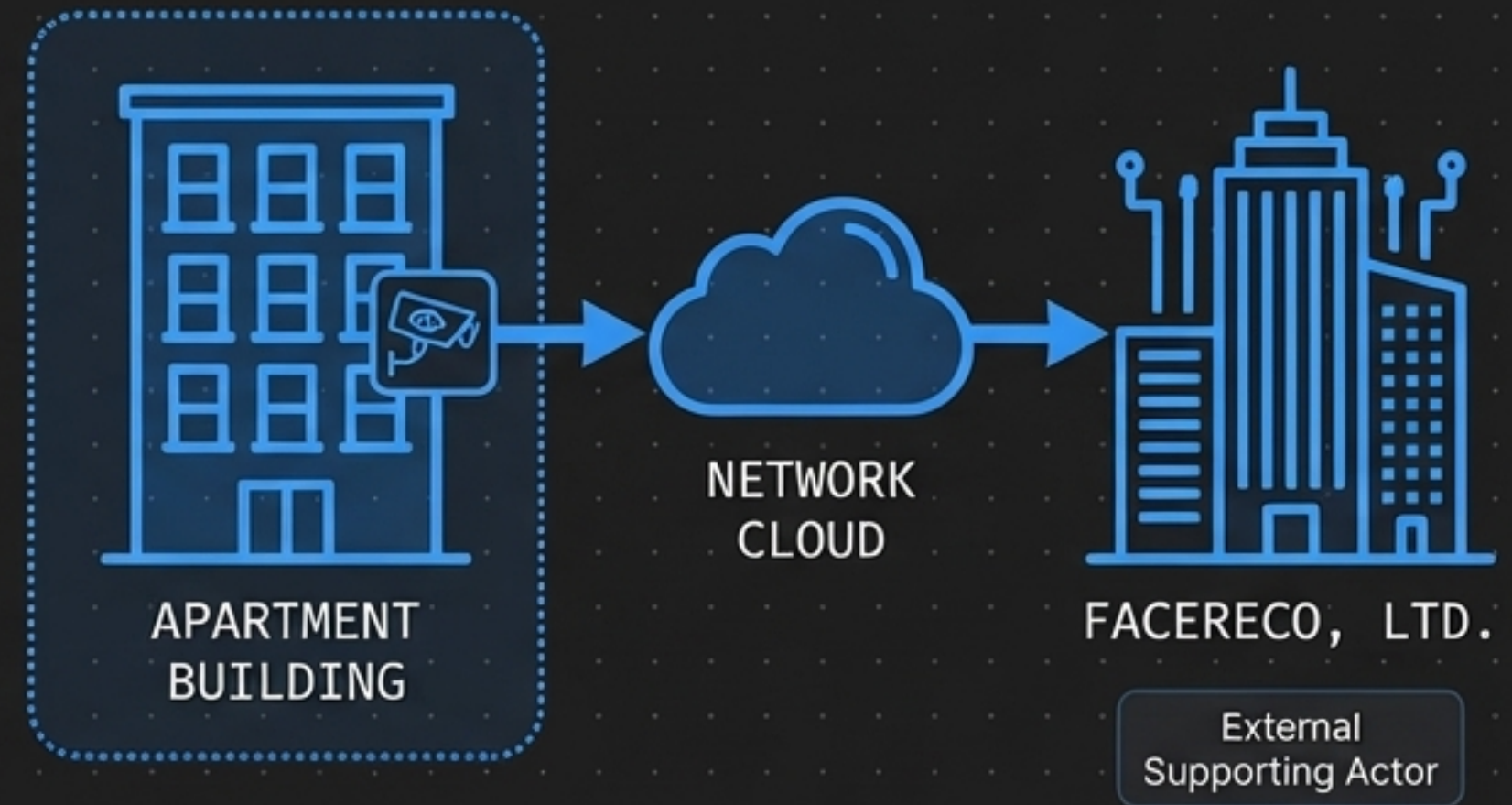
SYSTEM BOUNDARY & COMPONENT RELATIONSHIPS

Architectural Comparison: Local vs. Externalized Face Recognition

Case A: Local System Boundary

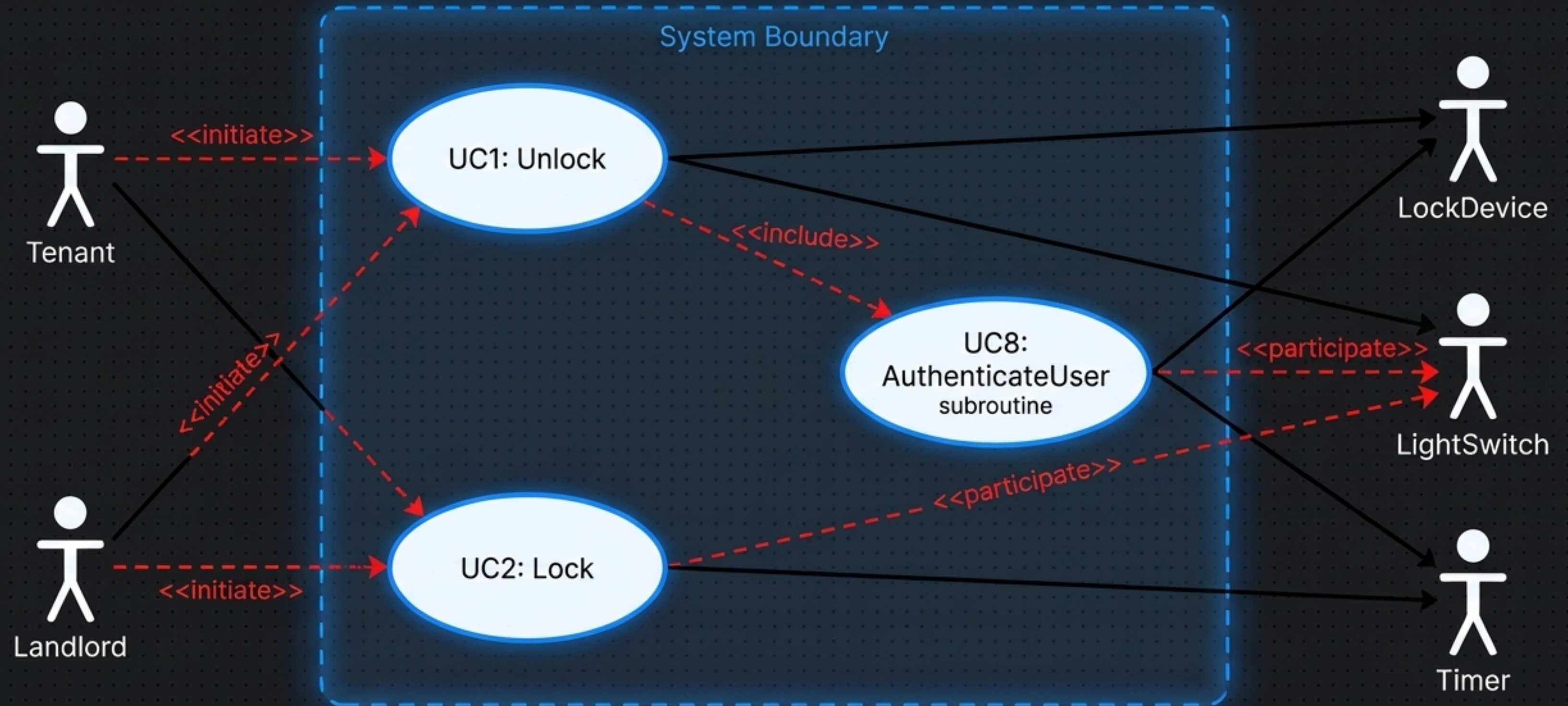


Case B: Externalized System Boundary

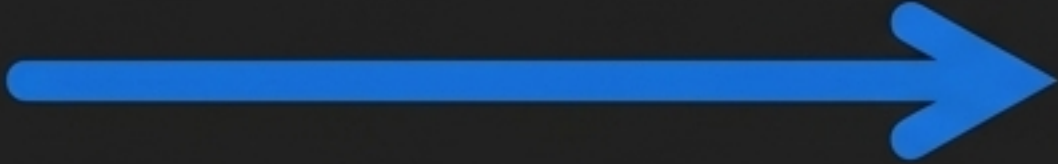
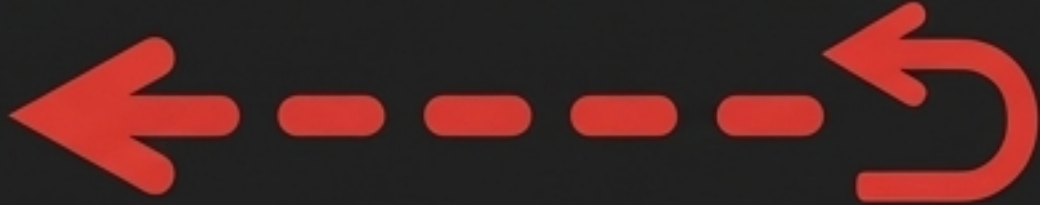


When **externalized, FaceReco, Ltd.** becomes an independent **Supporting Actor**, completely changing the internal Use Case definitions.

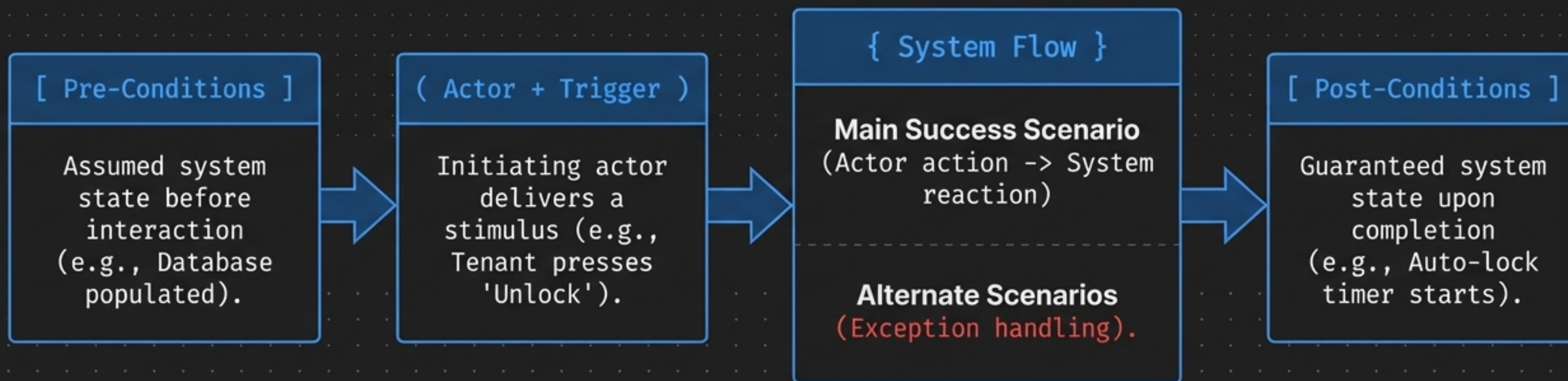
High-Level Architecture: Goal ARRIVE



Parametrizing Subroutines: <<include>> vs. <<extend>>

<<include>> (Mandatory)	<<extend>> (Optional / Conditional)
	
Traits: - Not optional. - Base use case is <u>incomplete</u> without it. ✓ - Execution is unconditional. - Does not change base behavior. UC1 (Unlock) <<include>> UC7 (AuthenticateUser). You <u>MUST</u> authenticate to unlock.	Traits: - Optional. - Base use case is <u>complete</u> without it. - Execution is conditional. - Changes base behavior. ManageAccount <<extend>> UC5 (InspectAccessHistory). You <u>MIGHT</u> check history while managing an account.

Use Case Function Pipeline



Use Case to Implementation Bridge

□ Bridging UML and Software Implementation

Flow of Events: UC-1 (Unlock)

1. Tenant selects "Unlock".



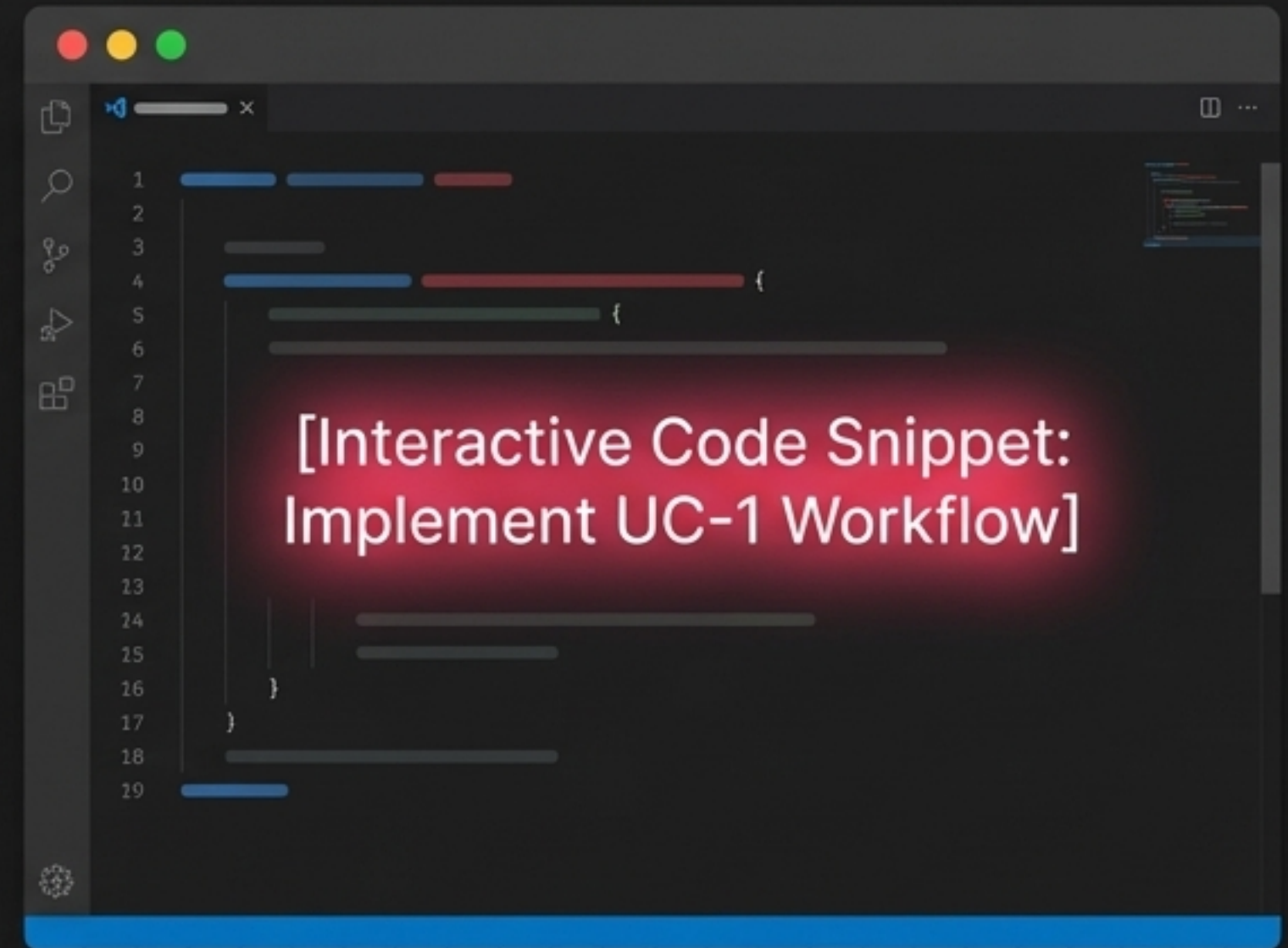
2. include::AuthenticateUser



3. System signals lock disarmed, turns light on.



4. System starts auto-lock timer.



Alternate Scenarios: UC-7 (AuthenticateUser)

2a. Tenant enters invalid key.

-> System marks failed attempt.

1a. System detects failed attempts > maximum allowed.

-> System sounds AlarmBell and notifies Police.

```
1 {  
2   < -----> {  
3     ( ----- {  
4  
5  
6  
7  
8  
9   }  
10  > -----> }  
11 }
```

[Interactive Code Snippet:
Implement Auth Loop &
Exception Handling]

Security & Risk Management

Risk Categorization

Intolerable

ALARP (As low as reasonably practical)

Acceptable

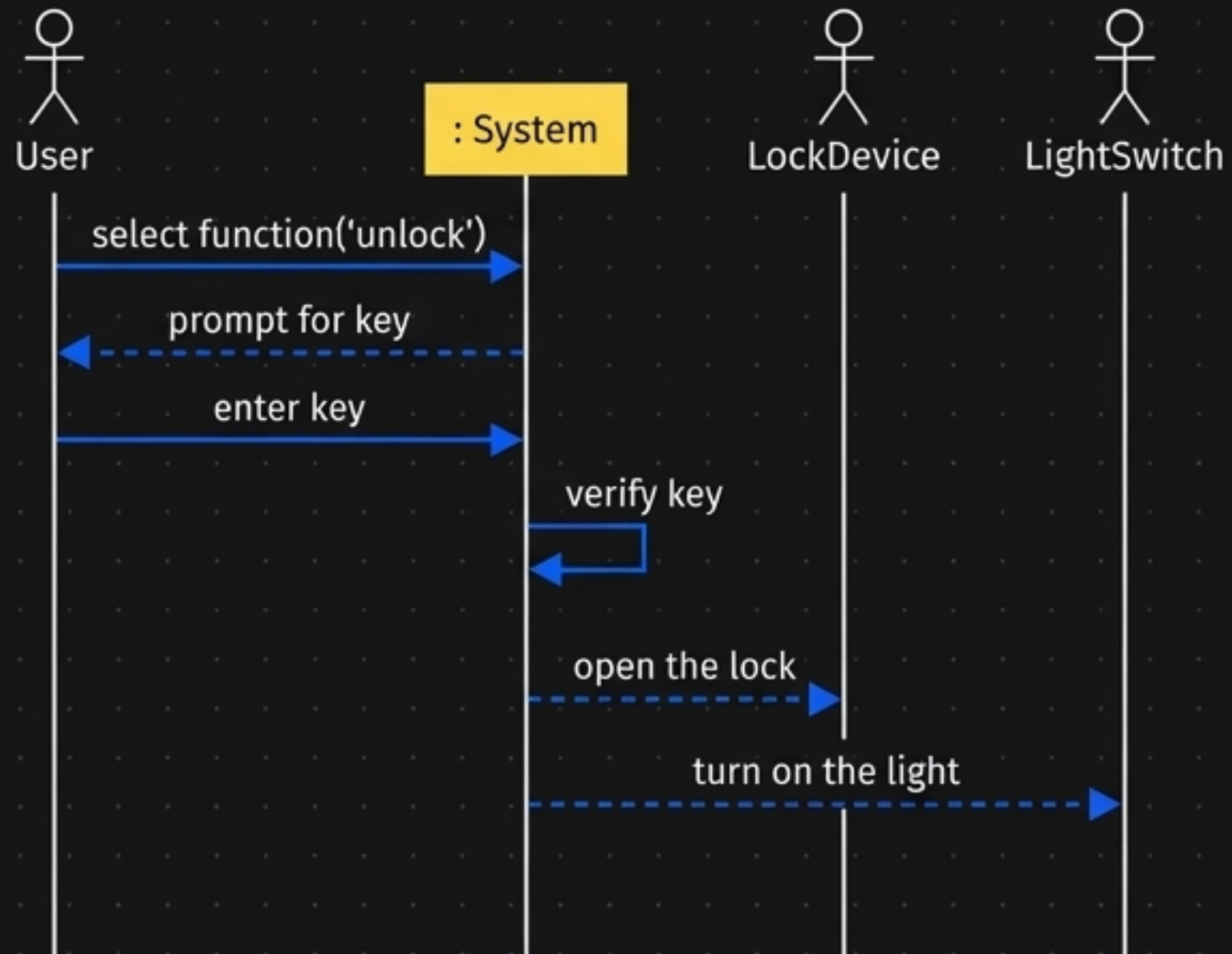
Modeling the Abuse Case

```
1 invalid-key, invalid-key... <= maxNumOfAttempts ;  
2 wait maxAttemptPeriod ;  
3 invalid-key, invalid-key...
```

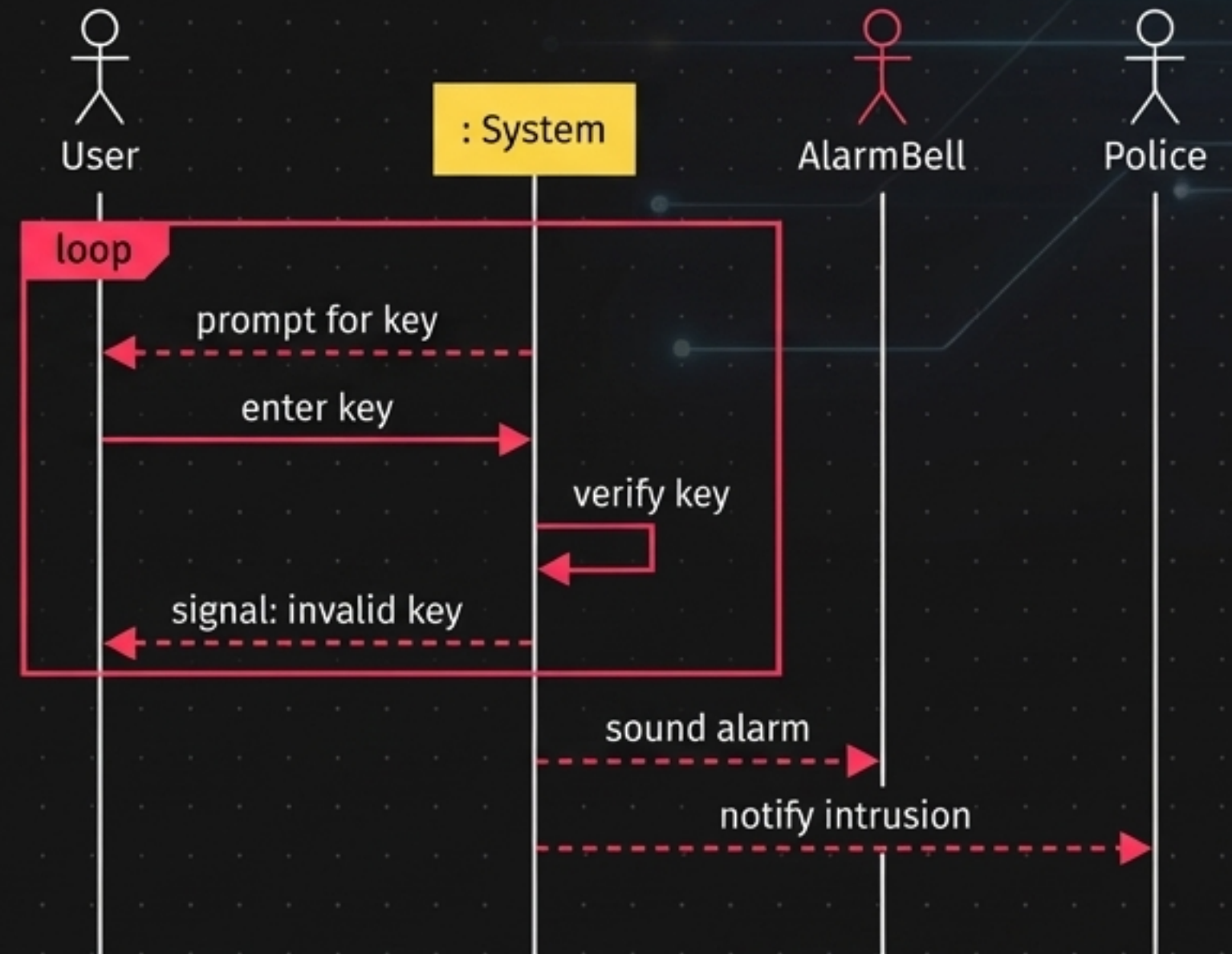
Requirements must explicitly dictate `maxNumOfAttempts` to transition a dictionary attack from an "Acceptable" risk to "Intolerable".

SYSTEM SEQUENCE DIAGRAMS: UNLOCK USE CASE

Happy Path

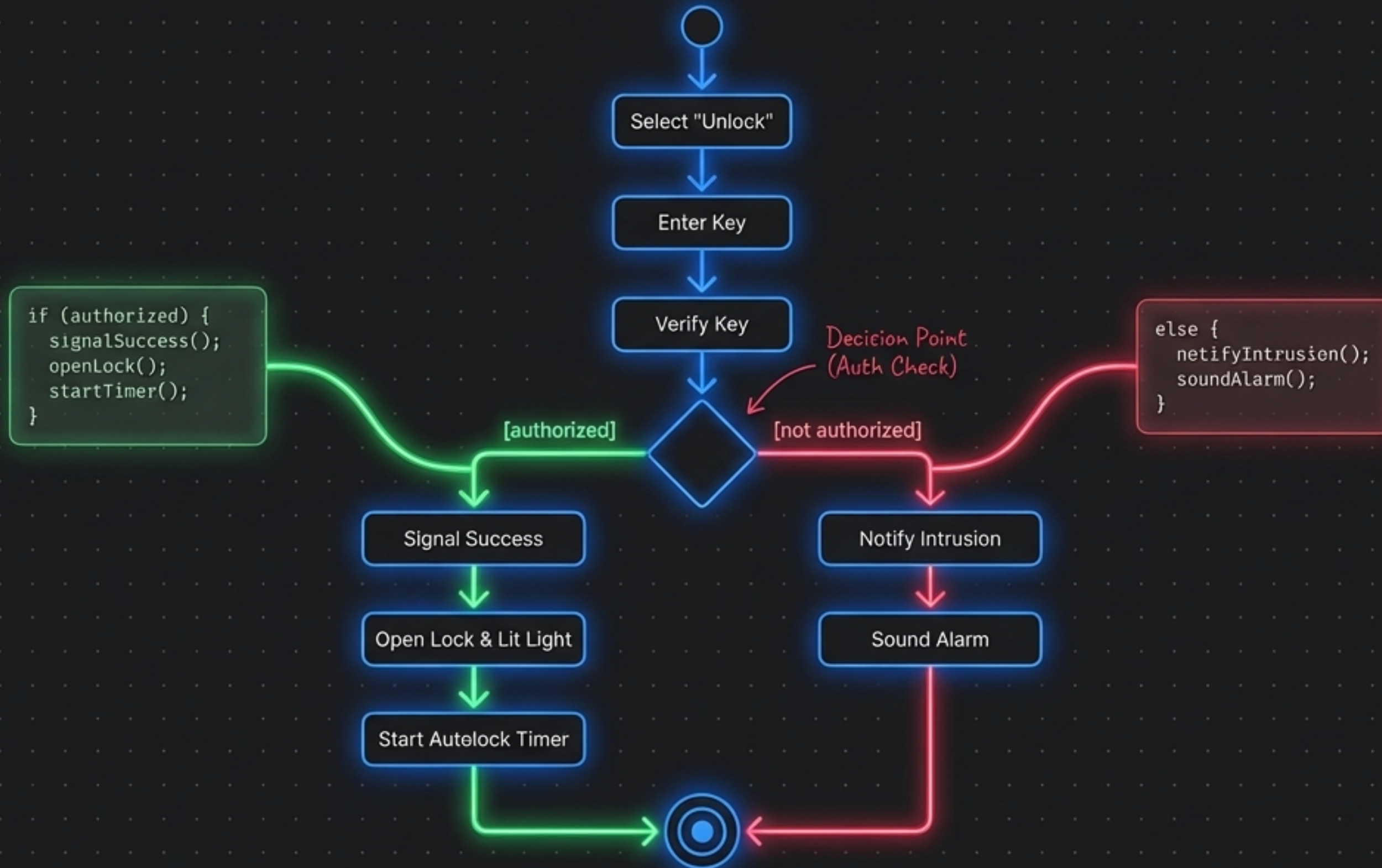


Failure Path



ACTIVITY DIAGRAM: UNLOCK WORKFLOW

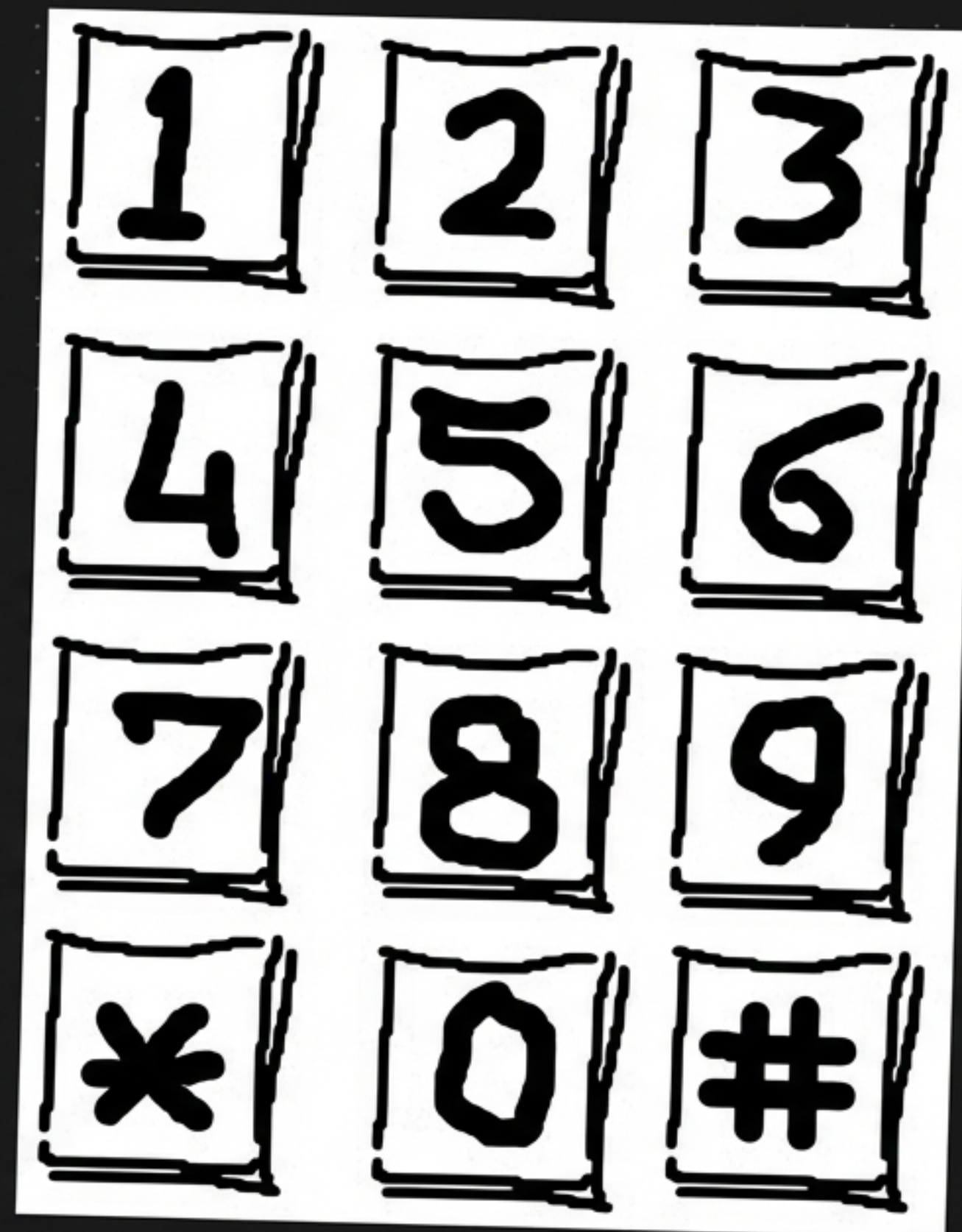
[Modeling System Workflows with Code Integration]



Phase 3: The Human Interface

Economize through constraint.

- ✓ Do not waste time with elaborate shading, coloring, or polished screenshots early on.
- ✓ Hand-drawing forces the team to focus purely on functional requirements, not aesthetics.
- ✓ Only prototype once the logical consensus is reached.



UI EVOLUTION MATRIX: DOOR KEYPAD HARDWARE ITERATION

V1 (NUMBERS ONLY)



Analysis:

First N digits validate.
Fails to allow correction for
unintentional mistypes.

V2 (ADD ACTION BUTTONS)



Analysis:

System reads LAST N digits
before 'Unlock' is pressed.
Allows users to correct typos.
Does not solve memory lapses
(*'Did I forget the code?'*).

V3 (ADD DISPLAY)



Analysis:

Solves memory lapses, but
introduces new engineering
constraints: Hardware cost,
weather sensitivity, and
security risks (*intruder can see
typed digits over the shoulder*).

Device Preferences

— □ ×

Activate for valid key

- ☒ Lights
- ☒ Music
- ☐ Air-conditioning
- ☐ Heating

Activate for burglary attempt

- ☒ Alarm bell
- ☒ Police
- ☐ Send SMS

Apply Close

(NOTE: Lock device is mandatory, not an option. Hardware constraints dictate software logic.)

Limitations of Use Cases

Often, the direct user (waiter, clerk, doctor) is acting on behalf of the actual beneficiary (the guest, the patient).

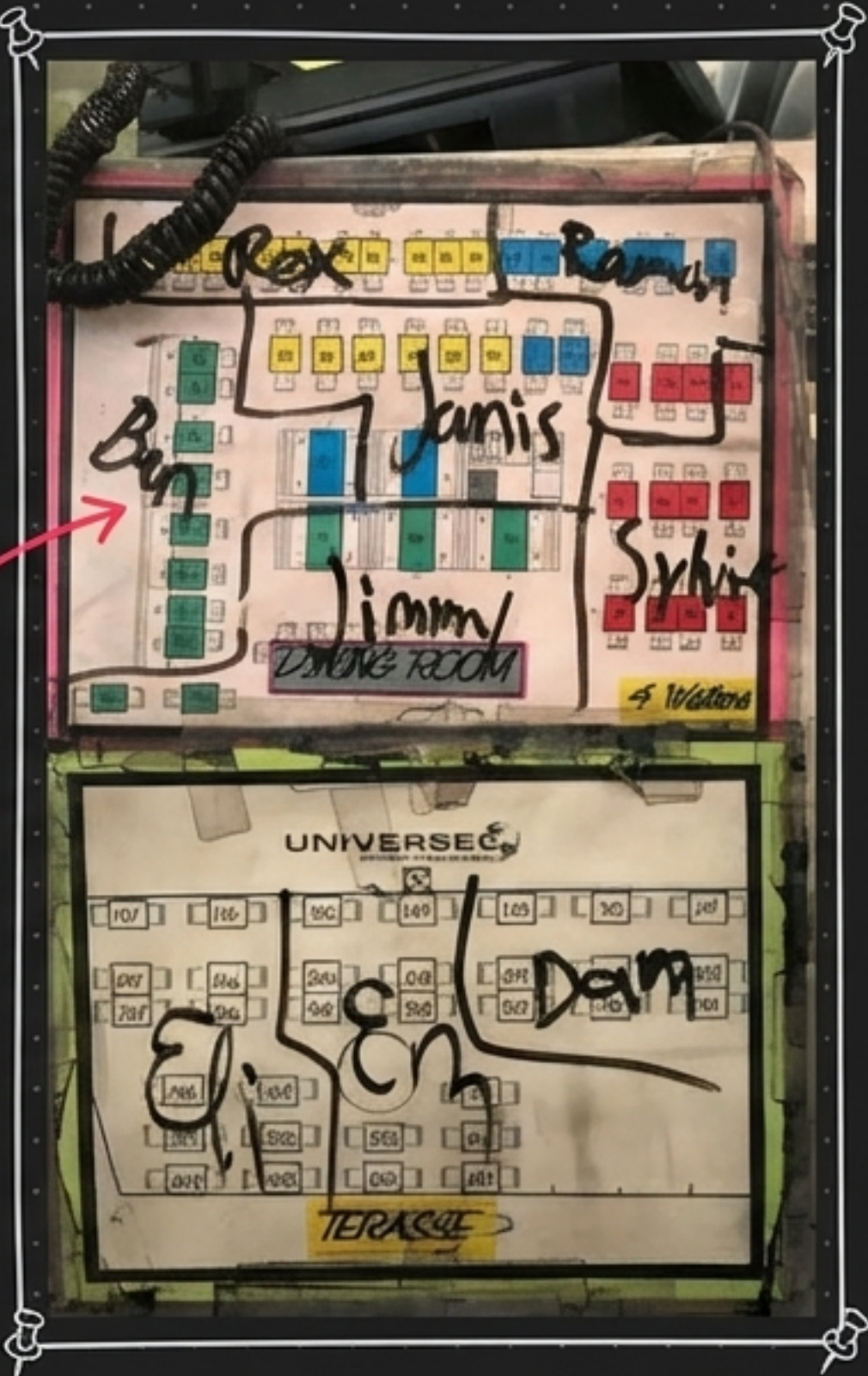
Whose business goals dictate the system? The user's efficiency, or the beneficiary's experience?

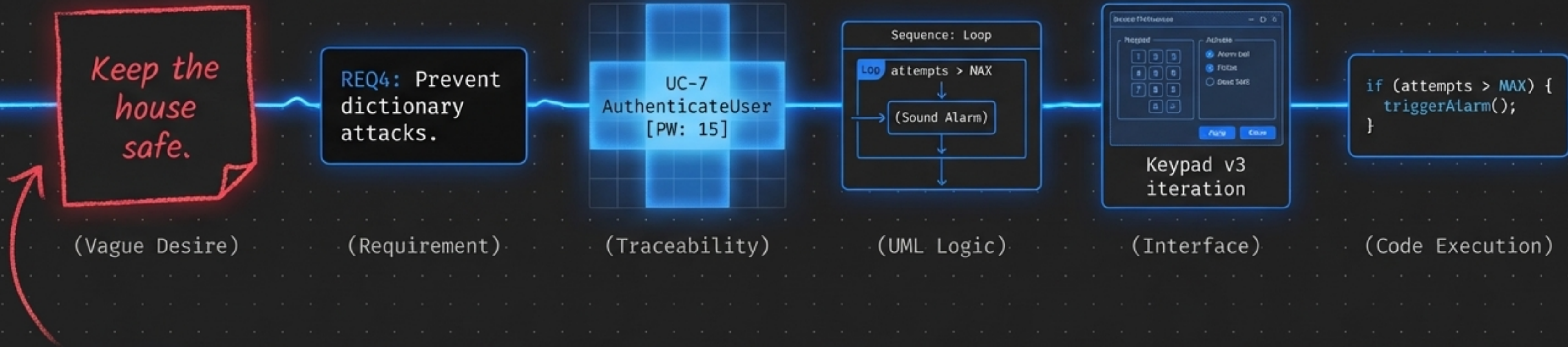
Insight

If you only model the intermediary, automation will eventually replace them entirely.

FIELD RESEARCH: RAW EVIDENCE

CHAOTIC
INTERMEDIARY
SYSTEM





Software Engineering is the rigorous translation of human intent into algorithmic reality.